Backend Node.js — NTI Courses Platform Exam

Express.js · MongoDB · Mongoose · JWT · Bcrypt · Joi · Multer
Time Allowed: 10–12 Hours **Total Marks:** 100

كل ال topic اتكلمنا عنها و شرحناها سوي فربنا معاكم و خلينا نشوف العظمه الي متاكد انكم هتعملوها 🥺🥺

Overview

You are tasked with building the **backend API** for a video courses platform. The system has three roles: **Admin**, **Teacher**, and **Student**. Teachers upload course content (video or PDF), students subscribe and watch sessions, and must pass quizzes to unlock the next session.

Roles & Permissions Summary

Action	Admin	Teacher	Student
Register / Login	✅	✅	✅
Manage all users	✅	❌	❌
Create a course	❌	✅	❌
Edit / Delete own course	❌	✅ (own only)	❌
Upload session (video/PDF)	❌	✅ (own course)	❌
Delete own session file	❌	✅ (own only)	❌
Subscribe to a course	❌	❌	✅
Stream a session video	❌	❌	✅ (subscribed)
View / download a PDF	❌	❌	✅ (subscribed)
Submit quiz answers	❌	❌	✅
Add questions to session	❌	✅ (own)	❌
Ban / unban a user	✅	❌	❌

---

1. `User`

- Fields to think about: `name`, `email`, `password` (hashed), `role` (`admin` | `teacher` | `student`), `isActive`
- **Hint:** The teacher has a `cardNumber` (the payment destination when a student subscribes to their course). Think about whether to embed it here or in a separate profile.
- **Hint:** You may want a `profilePicture` field if you support avatar upload.
- **Hint:** Timestamps are useful — use Mongoose `{ timestamps: true }`.

---

2. `Course`

- Fields to think about: `title`, `description`, `teacher` (ref → `User`), `price` (if `0` or `null` → free), `thumbnail`, `category`
- **Hint:** A course has many sessions. Should sessions be embedded or in a separate collection? Think about query patterns (streaming, quiz access) and decide.
- **Hint:** You'll need to track which students are enrolled. Consider a separate `Enrollment` model vs. an array of refs.

---

3. `Session`

- Fields to think about: `course` (ref → `Course`), `title`, `order` (integer, for sequential access), `contentType` (`video` | `pdf`), `filePath`, `duration`
- **Hint:** Each session has questions. Think carefully: embedded array of question objects, or a separate `Question` collection? Consider what happens when the teacher edits questions later.
- **Hint:** The student must pass the quiz of session `N` before accessing session `N+1`. Where do you store which sessions a student has completed?

---

4. `Question`

- Fields to think about: `session` (ref → `Session`), `text`, `options` (array of strings), `correctAnswerIndex`
- **Hint:** `correctAnswerIndex` should **never** be sent to the student in the API response. Use `.select('-correctAnswerIndex')` or a DTO pattern.
- **Hint:** A passing score threshold (e.g., 70%) can be stored on the Session or as a platform config. Decide where.

5. `Enrollment`

- Fields to think about: `student` (ref → `User`), `course` (ref → `Course`), `enrolledAt`, `completedSessions` (array of session IDs the student passed)
- **Hint:** This is the key model for access control — always check enrollment before streaming.
- **Hint:** Use a compound unique index on `{ student, course }` to prevent duplicate enrollments.

---

6. `Transaction`

- Fields to think about: `student` (ref → `User`), `teacher` (ref → `User`), `course` (ref → `Course`), `amount`, `cardNumber` (teacher's card), `status` (`pending` | `success` | `failed`), `createdAt`
- **Hint:** For this exam, you do not need a real payment gateway. Simulate the transaction: receive the card number and amount, set status to `success` automatically (or add basic validation logic).
- **Hint:** Only create an `Enrollment` after a `Transaction` with `status: success`.

File Upload — Multer Configuration

> **You must use `diskStorage`** — do **not** use `memoryStorage`.

Requirements:

- Videos allowed: `.mp4`, `.mkv`, `.webm`
- PDFs allowed: `.pdf`
- Destination folders should be separated: `uploads/videos/` and `uploads/pdfs/`
- File size limits: Videos → 500MB, PDFs → 20MB
- Filename should be unique — use `Date.now()` + original extension or `uuid`

Hint — Multer setup skeleton:

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    // decide folder based on file.mimetype
  },
  filename: (req, file, cb) => {
```

```
    // build a unique filename
  }
});

const fileFilter = (req, file, cb) => {
  // accept only allowed mimetypes
};

const upload = multer({ storage, fileFilter, limits: { fileSize: ... } });
```

Video Streaming with Byte-Range (Bounce Streaming)

One of the core requirements is **streaming video** using HTTP range requests. This allows the client to seek (skip forward/backward) inside a video without downloading the entire file.

How It Works:

1. Client sends a `GET /api/sessions/:id/stream` request with a `Range` header like: `Range: bytes=0-999999`
2. Server reads the file size, calculates the chunk, and responds with:
   - Status `206 Partial Content`
   - Headers: `Content-Range`, `Accept-Ranges`, `Content-Length`, `Content-Type`
3. Stream only the requested byte range using `fs.createReadStream({ start, end })`

Hint — Skeleton for the streaming handler:

```
export const streamVideo = async (req, res) => {
  // 1. Find session by ID, verify student is enrolled
  // 2. Check student passed quiz of PREVIOUS session (if not first)
  // 3. Get filePath from session document
  // 4. const stat = fs.statSync(filePath)  →  get total file size
  // 5. Parse req.headers.range  →  e.g. "bytes=0-"
  // 6. Calculate start, end, chunkSize
  // 7. Set headers and pipe the stream:
  //    res.writeHead(206, { ... })
  //    fs.createReadStream(filePath, { start, end }).pipe(res)
};
```

> If no `Range` header is sent, respond with `400 Bad Request` or stream from byte 0.

Joi Validation — Rules

You **must** validate all incoming request bodies with Joi. Below are the field rules:

Field	Rules
`email`	required, valid email format
`password`	required, min 8 chars, at least 1 uppercase, 1 number
`name`	required, min 2 chars, max 50 chars
`role`	must be one of `teacher`, `student` (admin created manually or via seed)
`price`	number, min 0
`cardNumber`	string, exactly 16 digits
`title` (course/session)	required, min 3 chars, max 100 chars
`order`	integer, min 1
`contentType`	must be `video` or `pdf`
`options` (question)	array, min 2 items, max 5 items
`correctAnswerIndex`	integer, must be valid index within options array

Authentication & Authorization

- Use **JWT** for access tokens. Token should contain: `{ _id, role }`.
- Token expiry: `7d`
- Use **bcrypt** with salt rounds = `10` for password hashing.
- Create two middlewares:
  - `authenticate` — verifies JWT, attaches `req.user`
  - `authorize(...roles)` — checks `req.user.role` against allowed roles

Example usage in routes:

```
router.delete('/courses/:id', authenticate, authorize('teacher'), deleteCourse);
```

Bounce Search (Fuzzy / Partial Search)

Implement a search endpoint on courses that supports **partial/fuzzy matching** on the course `title` and `description`.

- Use MongoDB `$regex` with the `i` (case-insensitive) flag.
- Support a `?q=` query parameter.
- Also support filtering by `?category=` and `?isFree=true/false`.
- Paginate results with `?page=` and `?limit=` (default limit: 10).

Hint:

```
const query = {};
if (req.query.q) {
  query.$or = [
    { title: { $regex: req.query.q, $options: 'i' } },
    { description: { $regex: req.query.q, $options: 'i' } }
  ];
}
```

All Required API Endpoints

Auth

```
POST   /api/auth/register          - Register (teacher or student)
POST   /api/auth/login             - Login, returns JWT
```

Users (Admin only)

```
GET    /api/users                  - List all users
GET    /api/users/:id              - Get user by ID
PATCH  /api/users/:id/ban          - Ban a user (set isActive: false)
PATCH  /api/users/:id/unban        - Unban a user
DELETE /api/users/:id              - Delete a user
```

Courses

```
POST   /api/courses                - Create course (teacher only) [multipart:
thumbnail]
GET    /api/courses                - List all courses (public) [?
q=&category=&isFree=&page=&limit=]
GET    /api/courses/:id            - Get course details (public)
```

```
PUT    /api/courses/:id            — Edit course (teacher, own only)
DELETE /api/courses/:id            — Delete course (teacher, own only)
GET    /api/courses/my             — Get own courses (teacher)
```

Sessions

```
POST   /api/courses/:courseId/sessions        — Add session (teacher, own
course) [multipart: file (video or pdf)]
GET    /api/courses/:courseId/sessions        — List sessions in a course
(enrolled student or teacher)
GET    /api/sessions/:id                      — Get session metadata
PUT    /api/sessions/:id                      — Edit session info (teacher,
own)
DELETE /api/sessions/:id                      — Delete session & file (teacher,
own)
GET    /api/sessions/:id/stream               — Stream video (byte-range,
enrolled student)
GET    /api/sessions/:id/pdf                  — Serve/download PDF (enrolled
student)
```

Questions

```
POST   /api/sessions/:sessionId/questions     — Add question (teacher, own
session)
GET    /api/sessions/:sessionId/questions     — Get questions for a session
(enrolled student, no correctAnswerIndex)
PUT    /api/questions/:id                     — Edit question (teacher, own)
DELETE /api/questions/:id                     — Delete question (teacher, own)
POST   /api/sessions/:sessionId/submit        — Submit quiz answers (student) →
returns pass/fail + unlocks next
```

Subscriptions & Payments

```
POST   /api/courses/:id/subscribe             — Subscribe to a course (student)
                                                → if price > 0: requires {
cardNumber } in body, creates Transaction
                                                → if price === 0: directly
creates Enrollment
GET    /api/enrollments/my                    — List student's enrolled courses
```

Admin Dashboard (Bonus)

```
GET    /api/admin/stats                        — Total users, courses,
transactions, revenue
```

Business Logic Rules

1. **Ownership check:** A teacher can only edit/delete their own courses and sessions. Verify `course.teacher.toString() === req.user._id.toString()`.
2. **Enrollment gate:** Before streaming a video or serving a PDF, confirm the student has an active `Enrollment` for that course. Return `403 Forbidden` otherwise.
3. **Sequential session lock:** Before allowing access to session `N` (where `N > 1`), confirm that the student's `completedSessions` in the `Enrollment` document includes the ID of session `N-1`.
4. **Quiz pass logic:** On `POST /submit`, calculate the score. If `(correct / total) >= 0.7` (70%), mark the session as completed in the `Enrollment`. Return the result clearly in the response.
5. **Free vs paid:** On the subscribe endpoint, check `course.price`. If `price` is `0` or `null`, skip payment and create `Enrollment` directly. Otherwise, validate `cardNumber` and create a `Transaction` first.
6. **Prevent duplicate enrollment:** Check if an `Enrollment` with `{ student, course }` already exists before creating a new one.

Submission Checklist

Before submitting, confirm:

- [ ] All listed API endpoints are implemented and reachable
- [ ] Joi validation is applied to every relevant route
- [ ] Multer `diskStorage` is used (not `memoryStorage`)
- [ ] JWT-based auth with role middleware on all protected routes
- [ ] Passwords are hashed with bcrypt before saving
- [ ] Video streaming uses `Range` header and returns `206 Partial Content`
- [ ] PDF sessions are served correctly (correct `Content-Type`)
- [ ] Quiz submission correctly blocks or unlocks next session
- [ ] Free courses bypass payment; paid courses require `cardNumber`
- [ ] Teacher cannot modify another teacher's content

- [ ] No `correctAnswerIndex` is exposed in student-facing question responses
- [ ] Search supports partial/case-insensitive matching with pagination
- [ ] `.env` file used for `JWT_SECRET`, `MONGO_URI`, `PORT`
- [ ] `README.md` included with setup instructions

Grading Breakdown

Section	Marks
Database schema design (models, relations, indexes)	20
Auth system (register, login, JWT, bcrypt, middleware)	15
Course & Session CRUD with Multer upload	20
Video streaming with Range header (byte-range)	15
Quiz system with sequential session locking	15
Subscription & Transaction logic (free vs paid)	10
Joi validation coverage	5
Total	**100**

Good luck 😈